



Compendio Pomodoro Python

🧠 Compendio Teorico — Pomodoro Focus Timer (Python + SOLID)

1. Introduzione al progetto

L'obiettivo dell'esercizio è costruire un **Pomodoro Focus Timer** applicando i principi **SOLID** in Python.

Il programma simula cicli di lavoro e pausa (es. 25/5 minuti) con un design **modulare, estendibile e testabile**.

Ogni parte del sistema ha **una sola responsabilità** e comunica tramite **interfacce leggere (Protocol o ABC)**.

Questo permette di cambiare comportamento (es. tipo di notifica o strategia di tempo) **senza modificare il codice esistente**.

2. Principi SOLID

Principio	Significato	Applicazione nel progetto
SRP — Single Responsibility Principle	Ogni classe deve avere una sola responsabilità .	<code>TimerService</code> conta i secondi, non invia notifiche.
OCP — Open/Closed Principle	Il codice deve essere aperto all'estensione , ma chiuso alla modifica .	Nuove strategie (<code>Classic25_5</code> , <code>Deep50_10</code>) senza toccare <code>Pomodoro</code> .
LSP — Liskov Substitution Principle	Ogni sottoclasse deve poter sostituire la superclasse.	Tutte le strategie rispettano l'interfaccia <code>ISessionStrategy</code> .
ISP — Interface Segregation Principle	Meglio interfacce piccole e specifiche .	<code>INotifier</code> ha solo <code>notify(message)</code> .
DIP — Dependency Inversion Principle	Le classi dipendono da astrazioni , non da implementazioni.	<code>Pomodoro</code> dipende da <code>INotifier</code> , non da <code>print()</code> .

3. Architettura a moduli

```
src/pomodoro/  
├── timer.py      → gestisce il tempo (SRP)  
├── notify.py     → gestisce le notifiche (ISP)  
├── sessions.py   → strategie (OCP)  
├── persistence.py → salvataggio CSV (SRP + DIP)  
├── core.py       → orchestratore principale (Pomodoro)  
└── runner.py     → entrypoint del programma
```

4. Il Timer (SRP + DIP)

Il **timer** deve scandire i secondi senza sapere nulla del resto del programma.

Usa un **TickProvider** iniettabile per gestire il tempo reale o finto (nei test).

```
class ITickProvider(Protocol):  
    def tick(self) → None: ...
```

Esempio di implementazione reale:

```
class RealTickProvider:  
    def tick(self) → None:  
        import time  
        time.sleep(1)
```

Esempio d'uso nel timer:

```
class TimerService:  
    def __init__(self, tick_provider=None):  
        self._tick = tick_provider or RealTickProvider()  
  
    def countdown(self, seconds, *, on_tick, on_completed):  
        # TODO: implementare il loop countdown  
        # on_tick(...) → ogni secondo  
        # on_completed() → alla fine  
        pass
```

Nei test, si potrà usare un **FakeTickProvider** che non dorme mai.

5. Le notifiche (ISP + DIP)

Il timer non deve scrivere direttamente su console.

Si introduce un'interfaccia `INotifier` per astrarre il canale di notifica.

```
class INotifier(Protocol):
    def notify(self, message: str) → None: ...
```

Implementazioni concrete:

```
class ConsoleNotifier:
    def notify(self, message: str) → None:
        print(message)

class BeepNotifier:
    def notify(self, message: str) → None:
        print("\a" + message) # emette un beep testuale
```

Grazie al DIP, `Pomodoro` potrà ricevere un qualsiasi `INotifier`.

6. Le strategie (OCP)

Ogni strategia definisce le durate di focus e break.

È un modo per **estendere** il comportamento senza toccare il core.

```
class ISessionStrategy(Protocol):
    def get_durations(self) → tuple[int, int]: ...
```

Strategie disponibili:

```
class Classic25_5:
    def get_durations(self):
        return 25*60, 5*60

class Deep50_10:
```

```
def get_durations(self):
    return 50*60, 10*60
```

E una versione **custom** per test e demo:

```
class Custom:
    def __init__(self, focus_sec, break_sec):
        # TODO: validare i parametri (>0)
        self.focus_sec = focus_sec
        self.break_sec = break_sec
```

7. Persistenza (SRP + DIP)

Serve a **registrare le sessioni completate** in un file CSV.

```
@dataclass
class SessionLog:
    started_at: datetime
    kind: Literal["focus", "break"]
    seconds: int
```

Interfaccia:

```
class ISessionRepository(Protocol):
    def save(self, log: SessionLog) → None: ...
```

Implementazione CSV:

```
class CsvSessionRepository:
    def __init__(self, path="sessions.csv"):
        # crea file e header se non esiste
        pass

    def save(self, log: SessionLog):
        # TODO: scrivere riga CSV con started_at.isoformat()
        pass
```

8. Pomodoro (Observer + DIP)

Il cuore del sistema: coordina timer, notifiche e persistenza.

- Riceve una **strategia** (`ISessionStrategy`).
- Esegue un ciclo **focus** → **break**.
- Alla fine di ogni fase:
 - notifica l'utente,
 - salva il log,
 - e opzionalmente chiama un **callback di evento** (`on_focus_completed`).

```
@dataclass
class Pomodoro:
    strategy: ISessionStrategy
    notifier: INotifier
    timer: TimerService
    repo: ISessionRepository
    on_focus_completed: Optional[Callable[[], None]] = None

    def run(self):
        focus_sec, break_sec = self.strategy.get_durations()

        # TODO:
        # 1. Countdown focus
        # 2. Notifica "Focus finito!"
        # 3. Salva SessionLog
        # 4. Chiama on_focus_completed (se definito)
        # 5. Countdown break + notifica "Break finito!"
        pass
```

9. Runner (CLI e Dependency Injection)

Il file `runner.py` serve come interfaccia da riga di comando:

- permette di scegliere una strategia,
- istanzia `Pomodoro` e lo esegue.

```
def choose_strategy() → ISessionStrategy:
    print("Scegli: 1=25/5, 2=50/10, 3=Custom breve (10/5)")
    c = input("Scelta: ").strip()
    if c == "1": return Classic25_5()
    if c == "2": return Deep50_10()
    return Custom(10, 5)

def main():
    strat = choose_strategy()
    pomodoro = Pomodoro(strat)
    pomodoro.run()
```

10. Test automatici (pytest)

Tutti i file in `tests/` controllano il comportamento del sistema.

Esempi di casi verificati:

- `TimerService` chiama `on_tick` e `on_completed`.
- `Pomodoro` notifica e salva i log.
- `Custom` valida i parametri.
- `CsvSessionRepository` scrive correttamente il file.

Esempio di test per il timer:

```
def test_timer_calls_callbacks():
    fake = FakeTick()
    t = TimerService(fake)
    called = []
    t.countdown(3, on_tick=lambda s: called.append(s), on_completed=lambda: called.append("done"))
    assert called[-1] == "done"
```

11. Suggerimenti extra ("wow")

Chi vuole completare con un tocco professionale può:

- aggiungere una **progress bar** con caratteri testuali,
 - colorare la console (`colorama`),
 - o estendere le strategie con pause multiple.
-

12. Checklist concettuale

- ✓ SRP rispettato (ogni file ha una sola responsabilità)
- ✓ DIP applicato (tutto dipende da interfacce, non da classi concrete)
- ✓ OCP rispettato (posso aggiungere nuove strategie e notifier)
- ✓ ISP applicato (interfacce piccole e mirate)
- ✓ Test pronti a validare l'implementazione